CONTRACTIQ: CONTRACT OBLIGATION TRACKING & COMPLIANCE MANAGEMENT PLATFORM

Sprint 3 – Database Design Document

Prepared By: Priyanka S
Project: ContractIQ – Contract Obligation Tracking & Compliance Management Platform
Sprint: Sprint 3 – Database Design
Technology Stack: PostgreSQL, SQLAlchemy, FastAPI

Table of Contents

1. Introduction

The ContractIQ platform is designed to simplify contract management by providing a centralized system for storing, tracking, and monitoring contracts throughout their lifecycle. The application enables organizations to efficiently manage contractual obligations, monitor renewals, generate reports, record user activities, and maintain complete audit trails.

A well-designed database is essential for ensuring data consistency, scalability, security, and efficient retrieval of information. This document presents the complete database design for the ContractIQ platform, including the required tables, attributes, primary keys, foreign keys, relationships, and the Entity Relationship (ER) Diagram.

2. Project Objective

The primary objective of this database design is to create a structured relational database that supports all major functionalities of the ContractIQ application.

The database has been designed to:

- Store contract information securely.
- Maintain multiple versions of contracts.
- Track contract obligations and compliance.
- Manage contract renewals.
- Generate reports.
- Record system activities.
- Maintain audit logs for accountability.
- Send notifications to users.
- Ensure data integrity using Primary Keys and Foreign Keys.

3. Database Design Overview

The ContractIQ database follows a relational database model where data is organized into multiple related tables. Each table represents a specific business entity within the application, and relationships between these entities ensure efficient data management while minimizing redundancy.

The design consists of nine major tables that work together to provide complete contract lifecycle management.

These tables are:

- Users
- Contracts
- Contract Versions
- Obligations
- Renewals
- Notifications
- Reports
- Audit Logs
- Activities

4. Database Schema

4.1 Users Table

Purpose

The Users table stores information about all registered users of the ContractIQ platform. Every individual who accesses the application is represented in this table. User information is referenced by multiple modules such as contracts, reports, notifications, audit logs, and activities.

Columns

Column Name	Data Type	Key	Description
id	Integer	Primary Key	Unique identifier for every user
full_name	Varchar(100)	—	Full name of the user
email	Varchar(255)	Unique	Email address used for login
password	Varchar(255)	—	Encrypted user password
role	Varchar(50)	—	User role (Admin, Manager, Employee)
is_active	Boolean	—	Indicates whether the account is active

4.2 Contracts Table

Purpose

The Contracts table stores the complete details of every contract managed by the system. It acts as the central table of the application because several other modules depend on contract information.

Columns

Column Name	Data Type	Key	Description
id	Integer	Primary Key	Unique contract ID
contract_name	Varchar(200)	—	Name of the contract
contract_number	Varchar(100)	—	Unique contract reference number
description	Text	—	Contract description
start_date	Date	—	Contract start date
end_date	Date	—	Contract expiry date
status	Varchar(50)	—	Current contract status
created_by	Integer	Foreign Key	References Users table

4.3 Contract Versions Table

Purpose

Contracts frequently undergo modifications during their lifecycle. This table stores every version of a contract, allowing users to maintain version history and access previous revisions whenever required.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
contract_id	Integer	Foreign Key
version_number	Integer	—
document_path	Varchar(255)	—
created_at	DateTime	—

4.4 Obligations Table

Purpose

The Obligations table stores all contractual responsibilities and compliance requirements associated with each contract. It enables organizations to monitor pending tasks and ensure contractual compliance.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
contract_id	Integer	Foreign Key
title	Varchar(200)	—
description	Text	—
due_date	Date	—
status	Varchar(50)	—

4.5 Renewals Table

Purpose

The Renewals table stores contract renewal information and allows organizations to track upcoming renewal dates and their current status.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
contract_id	Integer	Foreign Key
renewal_date	Date	—
renewal_status	Varchar(50)	—

4.6 Notifications Table

Purpose

The Notifications table stores alerts and reminders generated by the system. Notifications help users stay informed about upcoming deadlines, contract renewals, compliance activities, and other important events.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
user_id	Integer	Foreign Key
contract_id	Integer	Foreign Key
message	Text	—
sent_at	DateTime	—
is_read	Boolean	—

4.7 Reports Table

Purpose

The Reports table stores details of reports generated by users for contract monitoring, compliance tracking, and business analysis.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
generated_by	Integer	Foreign Key
report_name	Varchar(200)	—
report_type	Varchar(100)	—
generated_at	DateTime	—

4.8 Audit Logs Table

Purpose

The Audit Logs table maintains a permanent record of important actions performed by users within the application. It provides accountability, security, and traceability for administrative operations.

Columns

Column Name	Data Type	Key
id	Integer	Primary Key
user_id	Integer	Foreign Key
action	Varchar(255)	—
table_name	Varchar(100)	—
timestamp	DateTime	—

4.9 Activities Table

Purpose

The Activities table records routine user activities performed within the application. It helps administrators monitor user interactions and maintain an activity history.

Columns

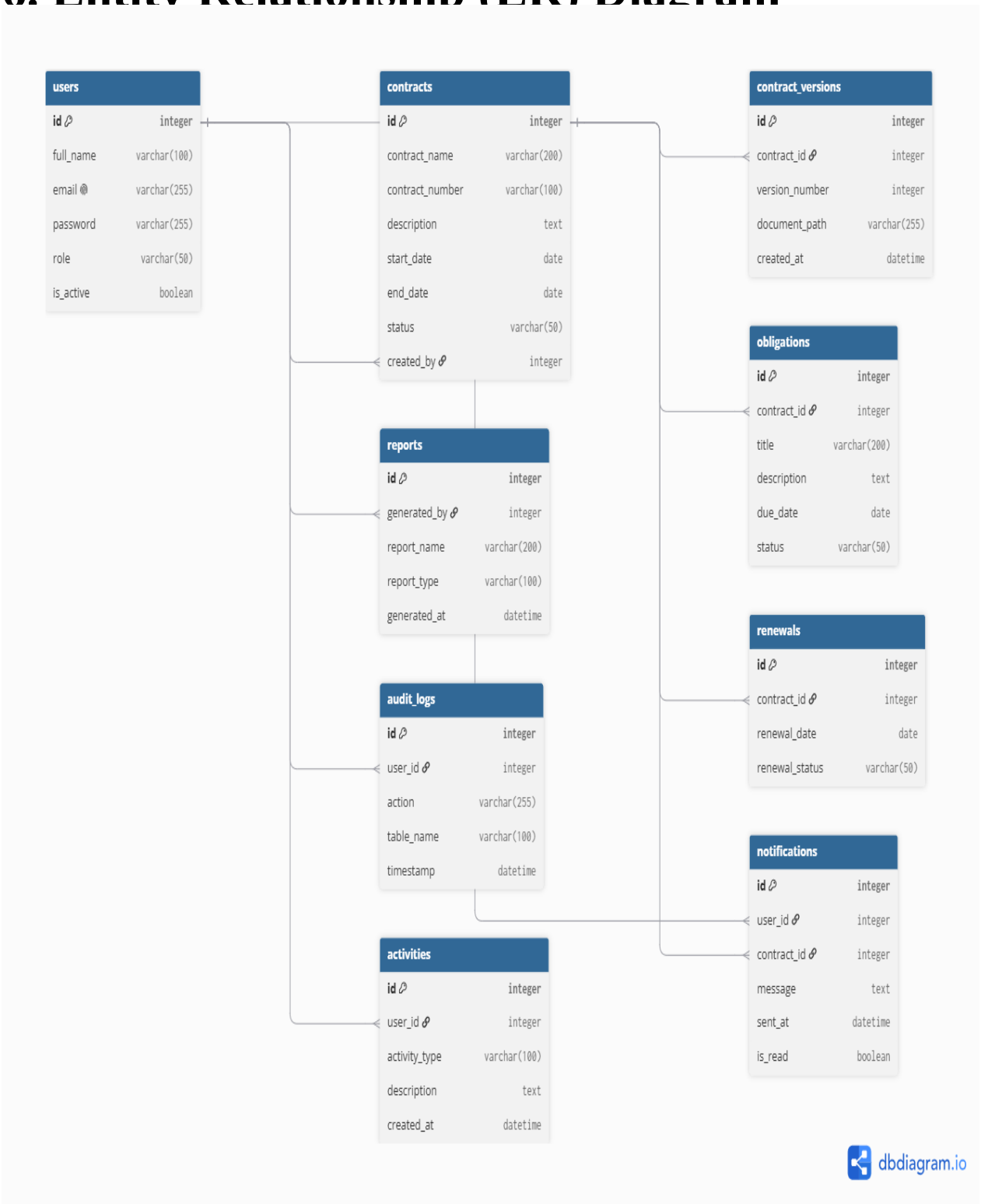
Column Name	Data Type	Key
id	Integer	Primary Key
user_id	Integer	Foreign Key
activity_type	Varchar(100)	—
description	Text	—
created_at	DateTime	—

5. Table Relationships

The ContractIQ database follows a relational model in which each table is connected using Primary Keys and Foreign Keys. These relationships ensure referential integrity while reducing duplicate data.

Parent Table	Child Table	Relationship
Users	Contracts	One-to-Many
Users	Reports	One-to-Many
Users	Notifications	One-to-Many
Users	Audit Logs	One-to-Many
Users	Activities	One-to-Many
Contracts	Contract Versions	One-to-Many
Contracts	Obligations	One-to-Many
Contracts	Renewals	One-to-Many
Contracts	Notifications	One-to-Many

6. Entity Relationship (ER) Diagram



7. Working of the Database

The ContractIQ database has been designed to support the complete lifecycle of contract management. Every user registered in the system can create and manage contracts based on their assigned role. Each contract is stored in the Contracts table and serves as the central entity for related business processes.

Whenever a contract is modified, a new record is stored in the Contract Versions table to maintain version history. Contractual responsibilities are managed through the Obligations table, while renewal schedules are tracked using the Renewals table. Notifications are generated to remind users of upcoming deadlines and contract-related events.

To support transparency and accountability, every important user action is recorded in the Audit Logs table, while routine activities are maintained in the Activities table. Reports generated by users are stored separately to facilitate monitoring and compliance analysis. The use of Primary Keys and Foreign Keys establishes strong relationships between tables, ensuring data integrity, consistency, and efficient retrieval of information.

8. Conclusion

The proposed database design provides a structured, scalable, and efficient foundation for the ContractIQ application. By organizing information into multiple related tables, the design minimizes redundancy while ensuring consistency through relational constraints.

The database supports all major functional modules, including contract management, version control, obligation tracking, renewal monitoring, reporting, notifications, auditing, and activity logging. This schema serves as a strong foundation for implementing SQLAlchemy models and backend APIs in the upcoming development sprints.